Multi-level LTS-RK4 for the graded L-shaped mesh

Implementation and validation log

Romain Mottier

August 23, 2026

Abstract

This report documents the design, implementation, and validation of a multi-level generalization of the explicit *Local Time-Stepping* RK4 scheme (LTS-RK4, "Algorithm 3") already present in DiSk++ (`erk_coupling_hho_scheme`), motivated by the prohibitive cost of the existing 2-level scheme on a graded L-shaped mesh with a reentrant corner singularity. The report covers: the diagnosis of the cost problem, two correctness bugs found in the pre-existing "restricted" code path, the new, validated restricted building blocks, the recursive multi-level generalization, and the current validation status.

Contents

1 Context and motivation

1.1 The cost problem

The L-shaped mesh graded toward the reentrant corner (interior angle $\omega = 3\pi/2$) is built via a coupled quadtree refinement: the background is uniformly refined ℓ times, and the corner gets an extra refinement depth $\text{depth} = 1.5(k+1)\,\ell$ (for HHO polynomial degree k), so as to recover the optimal h^{k+1} convergence rate despite the singularity (classical graded-mesh theory).

This construction makes the ratio $p = h_{\max}/h_{\min}$ explode: $p = 1, 32, 1024, 32768, \ldots$ for successive refinement levels $N = 0, 1, 2, 3, \ldots$ (for $k = 3$, p grows by a factor of 64 per level). The existing 2-level LTS-RK4 scheme performs p fine RK4 sub-steps per macro-step, and — critically — **every call touches the entire mesh**, not just the "fine" region. Total cost is therefore $O(p \times n_{\text{cells}})$, making $N = 3$ ($p \approx 32768$) roughly 9–10 hours of compute, and $N = 4$ entirely out of reach.

1.2 Grote's observation

A correctly implemented LTS-RK4 scheme should cost roughly the same as classical RK4 (no coarse/fine split) whenever the fine region is a minority of the degrees of freedom — which is precisely not what was observed. This motivated a two-part investigation:

1. make the 2-level scheme *genuinely* cost-restricted (every call should only touch the active band's dofs, not the whole mesh);

2. generalize to a true multi-level scheme, which turns out to be necessary because no single threshold can separate "fine is a minority" from "coarse is stable at the large time step" on a mesh whose cell-size distribution is nearly uniform in log-scale across 10–15 octaves (confirmed empirically, see §1.3).

1.3 Diagnostic: cell-size distribution

An octave-by-octave histogram of cell diameters ($h_{\max}/2^k$) shows that the uniformly-refined background (band 0) is the only genuinely large bucket; every subsequent octave, down to 15 octaves deep at $N = 3$, carries an almost constant cell count (~ 36), a direct consequence of the refinement "halo" margin added around the corner. This proves that **no single threshold** can give Grote's "fine is a minority" property without the opposite ("coarse") band itself spanning many octaves — which, as shown in §5.2, breaks the validity of the coarse role's polynomial approximation.

2 Empirical diagnosis of two pre-existing bugs

Before building anything new, the "restricted" code path already present in `erk_coupling_hho_scheme.hpp` (`build_LTS_subspaces` + `erk_weight_LTS_coarse_restricted` / `erk_weight_LTS_fine_restricted`, used by `ERK4_LTS_optimised.hpp`) was audited by code reading, then **empirically confirmed broken** by a controlled test: on a locally refined mesh (ratio $p = 2$), with a properly resolved time step (CFL sanity-checked via a $p = 1$ comparison, error ~ 0.02–0.10, plausible), the restricted path gives an error 10–30× *worse* despite a finer mesh — a clear correctness-bug signature, not an accuracy issue.

Bug A — coarse dofs never advanced. In `erk_weight_LTS_fine_restricted`, the RK4 update

$$\delta x = d\tau \cdot \tfrac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

is only scattered onto *fine* positions (`m_fine_active_c/f`); a comment claims coarse dofs are "handled elsewhere", but `erk_weight_LTS_coarse_restricted` never mutates the state at all — it only ever writes `w[]`. No other code path updates coarse dofs: they stay frozen at their initial value for the entire simulation.

Bug B — missing interface coupling term. In `compute_w_restricted`, the "face" output is only computed and scattered for *coarse-classified* faces. But an interface face (one coarse neighbour, one fine neighbour) is *always* classified fine by `assemble_P`'s union rule ("a face is fine if any adjacent cell is fine"). The coarse cell's contribution to that interface face — nonzero and physically required for the coupling — is therefore silently dropped.

Both bugs affect only `ERK4_LTS_optimised.hpp` and `ERK4_LTS_SSTAB.hpp`; they are **independent of** this session's L-shape prototypes, which use the unrestricted API (`erk_weight_LTS_coarse`/`erk_weight_L`) and remain trustworthy. On a minimum-risk principle, these pre-existing functions were *not* modified; new, correctly-designed functions were added instead (see §3).

3 New restricted building blocks (file `erk_coupling_hho_scheme.hpp`)

3.1 `LTS_subblock_set` and `build_LTS_subblock`

Unlike `build_LTS_subspaces` (a single coarse/fine sub-block pair in hardcoded class members, overwritten on every call), `build_LTS_subblock(own_c, own_f, halo_rings)` returns a self-contained `LTS_subblock_set` struct, allowing several bands to coexist — a hard requirement for multi-level.

The halo is built by breadth-first search over the cell–face adjacency graph implied by K_{fc}'s sparsity pattern: starting from `own_c`/`own_f`, each ring adds faces touching an already-included cell, then cells touching an already-included face. This halo guarantees the repeated-B-application chain (used by the "coarse" role, §3.2) stays exact, exploiting the fact that K_{cc} is exactly block-diagonal per cell in HHO (cells only couple through faces).

Each `LTS_subblock_set` stores *two* sets of restricted sub-matrices: `Kcc/Kcf/Kfc/Mc_inv/Sff_inv` (with halo, for the coarse role) and their `_own` counterparts (halo-free, for the fine role, which does not need one — see §3.3).

3.2 "Coarse" role: `erk_weight_LTS_coarse_v2`

Faithfully mirrors `erk_weight_LTS_coarse`'s algorithm, but restricted to `blocks.active_c/f` (own + halo): quadratic Lagrange interpolation of the force over $[0, dt]$, $B^i y_n$ and $B^i F_j$ chains (unmasked, using the halo to stay exact), then a *final* mask down to `own_c/own_f` only, right before the last B application — this final masking is what gives the output Taylor polynomial $w[0..3]$ its correct leak into interface faces (fixing Bug B), without contaminating the halo cells themselves (K_{cc} block-diagonal per cell + masking ⇒ zero contribution at non-own rows).

Output: $w_i(\tau) = w_0 + \tau w_1 + \frac{\tau^2}{2} w_2 + \frac{\tau^3}{6} w_3$, valid in local time over $[0, \text{len}]$ where len is this level's *own* interval length (see §5, not the global macro-step).

3.3 "Fine" role: `erk_weight_LTS_fine_v2`

One full RK4 step of size $d\tau$, with each stage $k = B(P_{\text{own}}y) + w(\tau)$ — the input y is masked down to `own` before applying B (exact mirror of $P_{\text{fine}} \cdot y_{\text{stage}}$ in the original algorithm), *but* B is applied using the halo-inclusive matrices, so the output is nonzero not just at own positions but also at directly-adjacent coarse (halo) cells — exactly matching the original unrestricted algorithm's behaviour, where a boundary coarse cell genuinely picks up a contribution via the K_{cf} coupling to the interface face. The update is scattered over the full `active_c/f` (own + halo), fixing Bug A.

3.4 Analytic fallback: `erk_weight_LTS_coarse_advance_uncovered`

Coarse cells *not* reached by the fine band's halo (the "deep interior" cells) receive no correction from the fine role at all. For them, $B(P_{\text{fine}}y) \equiv 0$ (no coupling to any fine face), so their exact whole-macro-step update is the closed-form integral of the cubic polynomial:

$$\Delta x = w_0\, dt + w_1 \tfrac{dt^2}{2} + w_2 \tfrac{dt^3}{6} + w_3 \tfrac{dt^4}{24}.$$

This is *exact*, not an approximation: Simpson's rule (what the RK4 sum reduces to when $B(P_{\text{fine}}y) = 0$ identically) is exact for a cubic polynomial. **Halo-covered and halo-uncovered cells must never both receive this update** — this was the source of a double-counting bug (and a missing-coverage bug in an earlier attempt), fixed and validated (§4).

4 Validation at $L = 2$

Methodology: direct, step-by-step comparison of the full state vector between the original (unrestricted, trusted) algorithm and the new v2 building blocks, on a real mesh with a *genuine* coarse/fine split (not a degenerate case where one band is empty). Two bugs were found and fixed during this validation before reaching an exact match:

1. A first attempt (`fine_v2` writing only its own positions, supplemented by an *unconditional* closed-form integral for *all* coarse-band own cells) gave a 1.3×10^{-5} mismatch after one step, growing to 0.05 after 141 steps — cause: boundary coarse cells then received *both* the $B(P_{\text{fine}}y)$ correction (implicitly, since unmasked in that version) *and* the closed-form integral, a partial double-count.

2. A second attempt (halo included in `fine_v2`, scattering over the full `active_c/f`, but *no* closed-form integral at all) gave a *worse* mismatch (10^{-3} after one step) — cause: coarse cells not reached by the halo (about 72/864 dofs in the tested case) then received *no* update at all.

3. The correct combination — closed-form integral applied *only* to coarse-band own cells not covered by the fine band's halo (checked by set membership) — gives an exact match to machine precision (9.7×10^{-16}, roundoff) over the full 141-step run.

5 Multi-level generalization

5.1 Recursive structure

At every level $i < L-1$: compute w_i (coarse role, using this level's *own* interval length len_i, not the global macro-step), combine with the ancestor contribution received as an argument (coefficient-wise sum, both expressed in the same local time frame), advance own cells not covered by the fine band's halo via the closed-form integral of the *combined* polynomial, then loop p_i times: at each sub-interval, Taylor-shift the combined polynomial to the new local origin, and either call `erk_weight_LTS_fine_v2` directly (base case, $i+1 = L-1$) or recurse.

Taylor shift. For a cubic polynomial $T(\tau) = w_0 + w_1\tau + \frac{w_2}{2}\tau^2 + \frac{w_3}{6}\tau^3$, the exact re-expansion around a new origin a is:

$$w_0' = T(a), \quad w_1' = T'(a), \quad w_2' = T''(a), \quad w_3' = w_3.$$

This is an exact identity (finite Taylor series of a degree-3 polynomial), not an approximation.

5.2 Key insight: per-level time-step scaling

A first attempt at applying §5 to a plain 2-level split, by aggressively shrinking the threshold h_c (to minimize the "fine" region) *while keeping the same global macro time step* $dt = \text{cfl} \cdot h_{\max}$, produced `NaN` — even at $h_c = 16\,h_{\min}$ (coarse band still spanning ~ 6 octaves). Diagnosis: the cubic polynomial $w[]$ has only 4 degrees of freedom; over a window dt calibrated for $h_{\max}$-scale cells, it cannot represent the dynamics of much smaller cells oscillating several times within that same window — an inherent breakdown of the method's validity, not an implementation bug (the coefficients themselves remain correct, as confirmed in §4).

Fix: every level i gets its own $dt_i = \text{cfl} \cdot \text{scale}_i$, where scale_i is that level's own characteristic cell size ($h_{\max}$ for level 0, the threshold $h_{c,i}$ for intermediate levels, $h_{\min}$ for the finest level). The branching ratio is $p_i = \text{round}(dt_i/dt_{i+1})$.

5.3 Validation of the generic recursion

The generic recursive driver, specialized to $L = 2$ with exactly the same threshold as §4's validation, reproduces the reference result to machine precision (6.9×10^{-16}). This confirms the recursion itself (Taylor shift, ancestor-contribution accumulation, analytic fallback) is correct — any gap seen at $L > 2$ is therefore a tuning (accuracy) question, not a correctness bug.

5.4 Results at $L = 5$ ($N = 2$, $k = 3$)

With bands spanning about 2 octaves each (5 levels total to cover $p = 1024$'s 10 octaves):

- No divergence (`NaN`) — confirms the time-step scaling fix.
- Wall time: 170 s, versus ~ 540 s for the original unrestricted scheme — a 3.2× speedup.
- Accuracy: L^2 error $= 3.7 \times 10^{-3}$, versus 1.14×10^{-5} for the reference scheme — a significant degradation ($\sim 326\times$), attributable to accumulated cubic approximations across 4 intermediate levels rather than a design flaw.

6 Current status and remaining work

- **Done and validated**: restricted building blocks (coarse/fine/fallback), generic recursive driver — both exact against the reference algorithm in their respective test cases.
- **Working but needs tuning**: the full multi-level scheme is stable and faster, but needs tuning (octaves per level, `cfl_factor`) to recover reference-level accuracy without losing the speed gain.
- **Next step**: a parametric study of the accuracy/speed trade-off, then apply to $N = 3$ and $N = 4$ to complete the 5-point convergence curve.